torch.linalg.svd#

https://pytorch.org/docs/stable/generated/torch.linalg.svd.html

Description:

Computes the singular value decomposition (SVD) of a matrix. Letting K\mathbb{K}K be R\mathbb{R}R or C\mathbb{C}C, the full SVD of a matrix A∈Km×nA \in \mathbb{K}^{m \times n}A∈Km×n, if k = min(m,n), is defined as where diag(S)∈Km×n\operatorname{diag}(S) \in \mathbb{K}^{m \times n}diag(S)∈Km×n, VHV^{\text{H}}VH is the conjugate transpose when VVV is complex, and the transpose when VVV is real-valued. The matrices UUU, VVV (and thus VHV^{\text{H}}VH) are orthogonal in the real case, and unitary in the complex case. When m > n (resp. m < n) we can drop the last m - n (resp. n - m) columns of U (resp. V) to form the reduced SVD: where diag(S)∈Kk×k\operatorname{diag}(S) \in \mathbb{K}^{k \times k}diag(S)∈Kk×k. In this case, UUU and VVV also have orthonormal columns.

Function Signature

```
torch.linalg.svd(A, full_matrices=True, *, driver=None,
out=None)#

Parameters

Keyword Arguments

Returns
```

Parameters

Keyword

driver (str, optional) – name of the cuSOLVER method to be used. This keyword argument only works on CUDA inputs. Available options are: None, gesvd, gesvdj, and gesvda. Default: None. out (tuple, optional) – output tuple of three tensors. Ignored if None.

Returns:

A named tuple (U, S, Vh) which corresponds to UUU, SSS, VHV^{\text{H}}VH above. S will always be real-valued, even when A is complex. It will also be ordered in descending order. U and Vh will have the same dtype as A. The left / right singular vectors will be given by the columns of U and the rows of Vh respectively.

Code Examples

```
>>> A = torch.randn(5, 3)
>>> U, S, Vh = torch.linalg.svd(A, full_matrices=False)
>>> U.shape, S.shape, Vh.shape
(torch.Size([5, 3]), torch.Size([3]), torch.Size([3, 3]))
>>> torch.dist(A, U @ torch.diag(S) @ Vh)
tensor(1.0486e-06)

>>> U, S, Vh = torch.linalg.svd(A)
>>> U.shape, S.shape, Vh.shape
(torch.Size([5, 5]), torch.Size([3]), torch.Size([3, 3]))
>>> torch.dist(A, U[:, :3] @ torch.diag(S) @ Vh)
tensor(1.0486e-06)

>>> A = torch.randn(7, 5, 3)
>>> U, S, Vh = torch.linalg.svd(A, full_matrices=False)
>>> torch.dist(A, U @ torch.diag_embed(S) @ Vh)
tensor(3.0957e-06)
```

Notes & Warnings

NOTE Note When full_matrices= True, the gradients with respect to U[..., :, min(m, n):] and Vh[..., min(m, n):, :] will be ignored, as those vectors can be arbitrary bases of the corresponding subspaces.

⚠ WARNING Warning The returned tensors U and V are not unique, nor are they continuous with respect to A. Due to this lack of uniqueness, different hardware and software may compute different singular vectors. This non-uniqueness is caused by the fact that multiplying any pair of singular vectors uk,vku_k, v_kuk,vk by -1 in the real case or by eiϕ,ϕ∈Re^{i \phi}, \phi \in \mathbb{R}eiϕ,ϕ∈R in the complex case produces another two valid singular vectors of the matrix. For this reason, the loss function shall not depend on this eiϕe^{i \phi}eiϕ quantity, as it is not well-defined. This is checked for complex inputs when computing the gradients of this function. As such, when inputs are complex and are on a CUDA device, the computation of the gradients of this function synchronizes that device with th

⚠ WARNING Warning Gradients computed using U or Vh will only be finite when A does not have repeated singular values. If A is rectangular, additionally, zero must also not be one of its singular values. Furthermore, if the distance between any two singular values is close to zero, the gradient will be numerically unstable, as it depends on the singular values σi\sigma_iσi through the computation of 1mini≠jσi2−σj2\frac{1}{\min_{i \neq j} \sigma_i^2 - \sigma_j^2}mini=jσi2−σj21. In the rectangular case, the gradient will also be numerically unstable when A has small singular values, as it also depends on the computation of 1σi\frac{1}{\sigma_i}σi1.

NOTE

See also torch.linalg.svdvals() computes only the singular values. Unlike torch.linalg.svd(), the gradients of svdvals() are always numerically stable. torch.linalg.eig() for a function that computes another type of spectral decomposition of a matrix. The eigendecomposition works just on square matrices. torch.linalg.eigh() for a (faster) function that computes the eigenvalue decomposition for Hermitian and symmetric matrices. torch.linalg.qr() for another (much faster) decomposition that works on general matrices.

Detailed Documentation

Documentation

Computes the singular value decomposition (SVD) of a matrix.

Letting K\mathbb{K}K be R\mathbb{R}R or C\mathbb{C}C, the full SVD of a matrix A∈Km×nA \in \mathbb{K}^{m \times n}A∈Km×n, if k = min(m,n), is defined as

where diag(S)∈Km×n\operatorname{diag}(S) \in \mathbb{K}^{m \times n}diag(S)∈Km×n, VHV^{\text{H}}VH is the conjugate transpose when VVV is complex, and the transpose when VVV is real-valued. The matrices UUU, VVV (and thus VHV^{\text{H}}VH) are orthogonal in the real case, and unitary in the complex case.

When m > n (resp. m < n) we can drop the last m - n (resp. n - m) columns of U (resp. V) to form the reduced SVD:

where diag(S)∈Kk×k\operatorname{diag}(S) \in \mathbb{K}^{k \times k}diag(S)∈Kk×k. In this case, UUU and VVV also have orthonormal columns.

Supports input of float, double, cfloat and cdouble dtypes. Also supports batches of matrices, and if A is a batch of matrices then the output has the same batch dimensions.

The returned decomposition is a named tuple (U, S, Vh) which corresponds to UUU, SSS, VHV^{\text{H}}VH above.

The singular values are returned in descending order.

The parameter full_matrices chooses between the full (default) and reduced SVD.

The driver kwarg may be used in CUDA with a cuSOLVER backend to choose the algorithm used to compute the SVD. The choice of a driver is a trade-off between accuracy and speed.

If A is well-conditioned (its condition number is not too large), or you do not mind some precision loss.

For a general matrix: ‘gesvdj’ (Jacobi method)

If A is tall or wide (m >> n or m << n): ‘gesvda’ (Approximate method)

If A is not well-conditioned or precision is relevant: ‘gesvd’ (QR based)

By default (driver= None), we call ‘gesvdj’ and, if it fails, we fallback to ‘gesvd’.

Differences with numpy.linalg.svd:

Unlike numpy.linalg.svd, this function always returns a tuple of three tensors and it doesn’t support compute_uv argument. Please use torch.linalg.svdvals(), which computes only the singular values, instead of compute_uv=False.

When full_matrices= True, the gradients with respect to U[…, :, min(m, n):] and Vh[…,

min(m, n):, :] will be ignored, as those vectors can be arbitrary bases of the corresponding subspaces.

The returned tensors U and V are not unique, nor are they continuous with respect to A. Due to this lack of uniqueness, different hardware and software may compute different singular vectors.

This non-uniqueness is caused by the fact that multiplying any pair of singular vectors uk,vku_k, v_kuk,vk by -1 in the real case or by eiϕ,ϕ∈Re^{i \phi}, \phi \in \mathbb{R}eiϕ,ϕ∈R in the complex case produces another two valid singular vectors of the matrix. For this reason, the loss function shall not depend on this eiϕe^{i \phi}eiϕ quantity, as it is not well-defined. This is checked for complex inputs when computing the gradients of this function. As such, when inputs are complex and are on a CUDA device, the computation of the gradients of this function synchronizes that device with the CPU.

Gradients computed using U or Vh will only be finite when A does not have repeated singular values. If A is rectangular, additionally, zero must also not be one of its singular values. Furthermore, if the distance between any two singular values is close to zero, the gradient will be numerically unstable, as it depends on the singular values σi\sigma_iσi through the computation of 1mini≠jσi2−σj2\frac{1}{\min_{i \neq j} \sigma_i^2 - \sigma_j^2}mini=jσi2−σj21. In the rectangular case, the gradient will also be numerically unstable when A has small singular values, as it also depends on the computation of 1σi\frac{1}{\sigma_i}σi1.

torch.linalg.svdvals() computes only the singular values. Unlike torch.linalg.svd(), the gradients of svdvals() are always numerically stable.

torch.linalg.eig() for a function that computes another type of spectral decomposition of a matrix. The eigendecomposition works just on square matrices.

torch.linalg.eigh() for a (faster) function that computes the eigenvalue decomposition for Hermitian and symmetric matrices.

torch.linalg.qr() for another (much faster) decomposition that works on general matrices.

A (Tensor) – tensor of shape (*, m, n) where * is zero or more batch dimensions.

full_matrices (bool, optional) – controls whether to compute the full or reduced SVD, and consequently, the shape of the returned tensors U and Vh. Default: True.

driver (str, optional) – name of the cuSOLVER method to be used. This keyword argument only works on CUDA inputs. Available options are: None, gesvd, gesvdj, and gesvda. Default: None.

out (tuple, optional) – output tuple of three tensors. Ignored if None.

A named tuple (U, S, Vh) which corresponds to UUU, SSS, VHV^{\text{H}}VH above.

S will always be real-valued, even when A is complex. It will also be ordered in descending order.

U and Vh will have the same dtype as A. The left / right singular vectors will be given by the columns of U and the rows of Vh respectively.